

PRACTICAL NO: 05

AIM:

Write a java program to perform methods and example on recursion.

THEORY:

Introduction to Recursion

Recursion is a programming technique in which a method calls itself repeatedly to solve a problem. A recursive method divides a large problem into smaller and simpler subproblems of the same type. The method continues calling itself until a specific condition, known as the **base condition**, is satisfied.

In Java, recursion is implemented by allowing a method to call itself either directly or indirectly. Every recursive method must contain two important parts:

1. **Base Condition:**

It is the condition that stops the recursive calls. Without a base condition, the method will continue calling itself indefinitely and may result in a `StackOverflowError`.

2. **Recursive Case:**

It is the part of the method where the method calls itself with a smaller or simpler input. This gradually moves the program toward the base condition.

Syntax of a Recursive Method

```
returnType methodName(parameters) { if (baseCondition) { return value; } return  
methodName(smallerInput); }
```

Example of Recursion

A common example of recursion is finding the **factorial of a number**.

The factorial of a positive integer n is defined as:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1$$

For example:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Using recursion, the factorial can be represented as:

$$\text{factorial}(n) = n \times \text{factorial}(n-1)$$

The base condition is:

$$\text{factorial}(0) = 1$$

For 5, the recursive calls work as:

$$\text{factorial}(5) = 5 \times \text{factorial}(4)$$

$$= 5 \times 4 \times \text{factorial}(3)$$

$$= 5 \times 4 \times 3 \times \text{factorial}(2)$$

$$= 5 \times 4 \times 3 \times 2 \times \text{factorial}(1)$$

$$= 5 \times 4 \times 3 \times 2 \times 1$$

$$= 120$$

Methods can be classified as:

- **Predefined methods:** Methods already provided by Java, such as `Math.sqrt()` and `System.out.println()`.
- **User-defined methods:** Methods created by the programmer according to the requirements.
- **Recursive methods:** User-defined methods that call themselves to solve a problem.

Advantages of Recursion

- It makes complex problems easier to understand and solve.
- It reduces the amount of code for problems that naturally have repetitive subproblems.
- It is useful for mathematical calculations such as factorial and Fibonacci series.
- It is commonly used in tree traversal, searching, sorting, and divide-and-conquer algorithms.

Disadvantages of Recursion

- Recursive calls require additional memory because each call is stored in the **call stack**.
- Incorrect or missing base conditions can cause infinite recursion.
- Excessive recursion can lead to `StackOverflowError`.
- For some problems, an iterative solution may be more efficient.



CODE SNIPPET:

```
import java.util.Scanner;

class Practical5 {

    // Recursive method to find factorial
    static int factorial(int num) {
        // Base case
        if (num == 0 || num == 1) {
            return 1;
        }

        // Recursive case
        return num * factorial(num - 1);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a number: ");
        int num = sc.nextInt();

        int result = factorial(num);

        System.out.println("Factorial of " + num + " = " + result);

        sc.close();
    }
}
```



Parul[®]
University

NAAC **A++**
ACCREDITED UNIVERSITY

COMPUTER SCIENCE AND ENGINEERING AIDS
FACULTY OF ENGINEERING & TECHNOLOGY
JPLAB(03010803PC04) B. Tech.3rd SEM
ENROLLMENT NO: -----

OUTPUT:

```
PS C:\Users\vishal gohil\Desktop\vishal java file> java Practical5
Enter a number: 5
Factorial of 5 = 120
PS C:\Users\vishal gohil\Desktop\vishal java file> █
```

PRACTICAL NO: 06

AIM:

Write a Java program to perform operations on One - Dimensional and Multi -Dimensional arrays.

THEORY:

An array is a collection of elements of the same data type stored under a single variable name. Arrays are used to store and process multiple values efficiently. In Java, arrays have a fixed size, and their elements are accessed using an index. The index starts from 0 and goes up to size - 1.

1. One-Dimensional Array

A one-dimensional array is a linear collection of elements arranged in a single row. It is useful for storing a list of values such as marks, numbers, or names.

Syntax:

```
dataType[] arrayName = new dataType[size];
```

Example:

```
int[] numbers = {10, 20, 30, 40, 50};
```

Here, numbers[0] contains 10, while numbers[4] contains 50.

Common operations performed on a one-dimensional array include:

- **Traversal:** Accessing and displaying each element.
- **Insertion:** Adding a value at a particular position.
- **Deletion:** Removing an element from an array.
- **Searching:** Finding a particular element.
- **Sorting:** Arranging elements in ascending or descending order.
- **Calculation:** Finding sum, average, maximum, and minimum values.

2. Multi-Dimensional Array

A **multi-dimensional array** is an array containing other arrays. The most commonly used type is a **two-dimensional (2D) array**, which represents data in the form of rows and columns, similar to a matrix or table.

Syntax:

```
dataType[][] arrayName = new dataType[rows][columns];
```

Example:

```
int[][] matrix = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Multi-dimensional arrays are commonly used for:

- Matrix operations
- Tables
- Student marks in different subjects
- Representing rows and columns of data
- Mathematical calculations

To access all elements of a 2D array, nested loops are generally used. The outer loop handles the rows, while the inner loop handles the columns.

Advantages of Arrays

- Arrays can store multiple values using a single variable.
- They provide fast access to elements using indexes.
- They make data processing easier using loops.
- They are useful for storing and manipulating large amounts of similar data.



CODE SNIPPET:

```
import java.util.Arrays;
import java.util.Scanner;

class Practical6 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // One-Dimensional Array
        System.out.print("Enter size of array: ");
        int n = sc.nextInt();

        int[] num = new int[n];

        System.out.println("Enter " + n + " elements:");
        for (int i = 0; i < n; i++) {
            num[i] = sc.nextInt();
        }

        // Display Array
        System.out.println("\nArray Elements:");
        System.out.println(Arrays.toString(num));

        // Sum, Maximum, Minimum
        int sum = 0;
        int max = num[0];
        int min = num[0];

        for (int i = 0; i < num.length; i++) {
            sum += num[i];

            if (num[i] > max)
                max = num[i];

            if (num[i] < min)
                min = num[i];
        }

        double average = (double) sum / num.length;

        System.out.println("Sum = " + sum);
        System.out.println("Maximum = " + max);
        System.out.println("Minimum = " + min);
        System.out.println("Average = " + average);

        // Sort Array
        Arrays.sort(num);

        System.out.println("\nSorted Array:");
        System.out.println(Arrays.toString(num));

        // Reverse Array
        System.out.print("Reverse Array: ");

        for (int i = num.length - 1; i >= 0; i--) {
            System.out.print(num[i] + " ");
        }

        // Search
        System.out.print("\n\nEnter element to search: ");
        int search = sc.nextInt();

        int position = Arrays.binarySearch(num, search);
```



```
if (position >= 0) {
    System.out.println(
        "Element " + search + " found at index " + position
    );
} else {
    System.out.println("Element " + search + " not found.");
}

// Multi-Dimensional Array
System.out.print("\nEnter number of rows: ");
int rows = sc.nextInt();

System.out.print("Enter number of columns: ");
int cols = sc.nextInt();

int[][] matrix = new int[rows][cols];

System.out.println("Enter matrix elements:");

for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        matrix[i][j] = sc.nextInt();
    }
}

// Display Matrix and Calculate Sum
int total = 0;

System.out.println("\nMatrix:");

for (int i = 0; i < rows; i++) {
    System.out.println(Arrays.toString(matrix[i]));

    for (int j = 0; j < cols; j++) {
        total += matrix[i][j];
    }
}

System.out.println("Sum of Matrix Elements = " + total);

// Transpose
System.out.println("\nTranspose of Matrix:");

for (int i = 0; i < cols; i++) {

    int[] transposeRow = new int[rows];

    for (int j = 0; j < rows; j++) {
        transposeRow[j] = matrix[j][i];
    }

    System.out.println(Arrays.toString(transposeRow));
}

sc.close();
}
```



OUTPUT:

```
PS C:\Users\vishal gohil\Desktop\vishal java file> java Practical6
Enter size of array: 5
Enter 5 elements:
2
5
6
3
3

Array Elements:
[2, 5, 6, 3, 3]
Sum = 19
Maximum = 6
Minimum = 2
Average = 3.8

Sorted Array:
[2, 3, 3, 5, 6]
Reverse Array: 6 5 3 3 2

Enter element to search: 5
Element 5 found at index 3

Enter number of rows: 5
Enter number of columns: 2
Enter matrix elements:
55
6
2
3
4
5
3
2
1
2

Matrix:
[55, 6]
[2, 3]
[4, 5]
[3, 2]
[1, 2]
Sum of Matrix Elements = 83

Transpose of Matrix:
[55, 2, 4, 3, 1]
[6, 3, 5, 2, 2]
PS C:\Users\vishal gohil\Desktop\vishal java file> |
```

PRACTICAL NO: 07

AIM:

Write a Java program to demonstrate 1D arrays and 2D arrays.

- A) Maximum value and second maximum value without duplicates
- B) Sort names in ascending order
- C) Addition of two matrices
- D) 3×3 matrix multiplication

THEORY:

Arrays in Java

An array in Java is a collection of elements of the same data type stored under a single variable name. Arrays are used to store and process multiple values efficiently.

There are mainly two types of arrays used in this practical:

1. 1D Array

A one-dimensional (1D) array stores elements in a single row or list

2. 2D Array

A two-dimensional (2D) array stores elements in rows and columns, similar to a matrix.

A) Maximum and Second Maximum Without Duplicates

In this operation, we find the **largest and second-largest distinct values** from a 1D array.

Logic:

- Initialize max and secondMax.
- Traverse the array.
- If the current element is greater than max, update both values.
- If the element is smaller than max but greater than secondMax, update secondMax.
- Ignore duplicate values of max.

B) Sort Names in Ascending Order

A 1D String array can be sorted alphabetically in ascending order.

Logic:

- Store names in a String array.
- Compare and arrange the names alphabetically.
- The names are displayed in ascending order.

C) Addition of Two Matrices

Matrix addition is performed using 2D arrays. Two matrices can be added only when they have the same number of rows and columns.

For two 3×3 matrices:

$$C[i][j] = A[i][0] \times B[0][j] + A[i][1] \times B[1][j] + A[i][2] \times B[2][j]$$

D) 3×3 Matrix Multiplication

Matrix multiplication is performed by multiplying the rows of the first matrix with the columns of the second matrix.

Logic:

- Use three nested loops.
- The first loop selects the row of the first matrix.
- The second loop selects the column of the second matrix.
- The third loop performs multiplication and addition.
- Store the result in the third matrix.



Code Snippet:

```
import java.util.Arrays;
import java.util.Scanner;

class Practical7 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // =====
        // A) Maximum and Second Maximum in 1D Array
        // =====

        System.out.print("Enter size of array: ");
        int num = sc.nextInt();

        int[] arr = new int[num];

        System.out.println("Enter array elements:");

        for (int i = 0; i < num; i++) {
            arr[i] = sc.nextInt();
        }

        int max = arr[0];
        int secondMax = Integer.MIN_VALUE;

        for (int i = 1; i < num; i++) {

            if (arr[i] > max) {
                secondMax = max;
                max = arr[i];
            }

            else if (arr[i] > secondMax && arr[i] != max) {
                secondMax = arr[i];
            }
        }

        System.out.println("Maximum value = " + max);

        if (secondMax == Integer.MIN_VALUE) {
            System.out.println("Second Maximum does not exist");
        }
        else {
            System.out.println("Second Maximum value = " + secondMax);
        }

        // =====
        // B) Sorting Names in Ascending Order
        // =====

        System.out.print("\nEnter number of names: ");
        int size = sc.nextInt();

        String[] names = new String[size];

        System.out.println("Enter names:");

        for (int i = 0; i < size; i++) {
            names[i] = sc.next();
        }

        Arrays.sort(names);

        System.out.println("Names in ascending order:");

        for (int i = 0; i < size; i++) {
            System.out.println(names[i]);
        }
    }
}
```



```
// =====  
// C) Addition of Two 2x2 Matrices  
// =====  
  
int[][] a = new int[2][2];  
int[][] b = new int[2][2];  
int[][] sum = new int[2][2];  
    System.out.println("\nEnter elements of first 2x2 matrix:");  
  
    for (int i = 0; i < 2; i++) {  
        for (int j = 0; j < 2; j++) {  
            a[i][j] = sc.nextInt();  
        }  
    }  
    System.out.println("Enter elements of second 2x2 matrix:");  
  
    for (int i = 0; i < 2; i++) {  
        for (int j = 0; j < 2; j++) {  
            b[i][j] = sc.nextInt();  
        }  
    }  
    System.out.println("Addition of matrices:");  
  
    for (int i = 0; i < 2; i++) {  
        for (int j = 0; j < 2; j++) {  
            sum[i][j] = a[i][j] + b[i][j];  
            System.out.print(sum[i][j] + " ");  
        }  
    }  
    System.out.println();  
}  
// =====  
// D) Multiplication of 3x3 Matrices  
// =====  
  
int[][] matrix1 = new int[3][3];  
int[][] matrix2 = new int[3][3];  
int[][] result = new int[3][3];  
  
    System.out.println("\nEnter first 3x3 matrix:");  
  
    for (int i = 0; i < 3; i++) {  
        for (int j = 0; j < 3; j++) {  
            matrix1[i][j] = sc.nextInt();  
        }  
    }  
  
    System.out.println("Enter second 3x3 matrix:");  
  
    for (int i = 0; i < 3; i++) {  
        for (int j = 0; j < 3; j++) {  
            matrix2[i][j] = sc.nextInt();  
        }  
    }  
    // Matrix multiplication  
    for (int i = 0; i < 3; i++) {  
        for (int j = 0; j < 3; j++) {  
            for (int k = 0; k < 3; k++) {  
                result[i][j] += matrix1[i][k] * matrix2[k][j];  
            }  
        }  
    }  
    System.out.println("Multiplication of matrices:");
```



OUTPUT:

```
PS C:\Users\vishal gohil\Desktop\vishal java file> java Practical7
Enter size of array: 3
Enter array elements:
6 8 6
Maximum value = 8
Second Maximum value = 6

Enter number of names: 3
Enter names:
vishal bhimbhai gohil
Names in ascending order:
bhimbhai
gohil
vishal

Enter elements of first 2x2 matrix:
2 6
2 5
Enter elements of second 2x2 matrix:
2 6
3 8
Addition of matrices:
4 12
5 13

Enter first 3x3 matrix:
3 6 8
5 6 3
5 9 8
Enter second 3x3 matrix:
5 7 6
5 9 8
8 4 3
Multiplication of matrices:
109 107 90
79 101 87
134 148 126
PS C:\Users\vishal gohil\Desktop\vishal java file> |
```